Recommender systems: from ratings to factors

MovieLens 1M

LECTURE 21 LECTURE NOTES

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Retrieval, with the query removed

	Retrieval	Recommendation
Input	a query and a corpus	a **user** and a catalogue
Output	a ranking	a ranking
Signal	the query text	what this user did before
Labels	judged by assessors, deliberately	a **by-product of use**

The output is the same shape, so Lecture 19's metrics apply unchanged. The *input* is what differs, and the last row is what makes this hard: nobody rated a film in order to help you.

Why this is worth two lectures

- It is the machine learning most people actually meet — a feed, a shop, a music service — and mostly without noticing
- It is where the evaluation habits of this course are most often abandoned, and the literature has had public reproducibility failures to prove it
- And it is where the whole course converges: Lecture 8's SVD, Lecture 5's regularisation, Lecture 19's ranking metrics and Lecture 20's two-tower architecture all appear again, doing the same jobs

If you take one thing from Part V into a job, it will probably be from these two lectures.

One dataset, one prize, one lesson

The methods in this lecture came out of a public competition with a million-dollar prize and a fixed test set. Three things came out of it that survived the models:

- **Biases matter more than anyone expected**, and modelling them explicitly was worth more than most architectural cleverness
- **Ensembles win competitions and are not deployed** — the winning blend was never put into production, because its cost exceeded its value
- **A single fixed test set gets overfitted** by a community, not just by a model — hundreds of teams selecting on one leaderboard is a search procedure

The third is Lecture 3's point about a test set used more than once, at the scale of a whole field.

Today

1. The problem, the data, and the long tail (15 min)
2. Baselines: how much of a rating is the user, and how much the film (15 min)
3. **Derivation 21** — matrix factorisation, and why the SVD of Lecture 8 cannot simply be taken (35 min)
4. Neighbourhood methods, and implicit feedback (20 min)

What you should be able to do afterwards

- State the recommendation problem, and how its data differs from a labelled dataset
- Write down the factorisation objective, and say why the sum is over observed entries only
- Explain precisely why the SVD cannot be applied to a matrix with missing entries — and what happens if you fill them anyway
- Derive the ALS half-step, and say why it is a ridge regression
- Say what a bias term is doing, and how much of the signal it carries
- Explain why implicit feedback has no negatives

FIRST FIFTEEN MINUTES

The problem, and the data

15 minutes

The task

6,040 users. 3,706 films. 1,000,209 ratings on a 1–5 scale.

Two questions, and they are not the same question:

- **Rating prediction** — what would this user give this film? Scored by RMSE. This is Lecture 21
- **Top-N recommendation** — which ten films should we show? Scored by the ranking metrics of Lecture 19. This is Lecture 22

> **THEY COME APART, AND THAT IS THE SUBJECT OF THE NEXT LECTURE**
>
> A model that predicts every rating within half a star can still recommend badly, because the films a user would rate highly are not the films they have not seen.

Why MovieLens, and what it is not

- **Explicit ratings**, which most systems do not have — but without them, rating prediction cannot be taught at all
- **Timestamps**, which is why this release and not the smaller one: half of Lecture 22 is the difference between splitting a history at random and splitting it in time
- **1,000,209 ratings** — large enough that the results are not noise, small enough to factorise on a CPU in seconds

> **AND WHAT IT IS NOT**
>
> It is a *research dataset from a system that asked people to rate films*. It is not a log of a production recommender, its users are self-selected, and its sparsity pattern is unlike a real catalogue's. Everything measured here transfers as a **method**, not as a number.

The matrix, and what is not in it

Users × films	22.4 million cells
Ratings present	1,000,209
Density	**4.5%**
Ratings per user, median	96
Ratings per film, median	124

95.5% of the matrix is **missing**, and this is the central fact of the lecture.

✗ Missing is not zero. A cell is empty because the user never saw the film, or saw it and did not rate it, or was never shown it by whatever system was running at the time. Every one of those is a different meaning, and none of them is "disliked".

Not missing at random

Worse: the pattern of what is missing is *informative*. People rate films they chose to watch, and they chose them because they expected to like them.

- A horror film unrated by a user who hates horror is missing **because** they hate horror
- Which is a rating, of a kind — and it is not in the data
- Every model in this lecture trains on observed entries and is evaluated on observed entries, so it never sees this

> **THE SAME SHAPE AS LECTURE 19'S POOLING**
>
> The data records what someone chose to look at, and the choice was not random. In retrieval it made recall a lower bound; here it biases the training set itself.

What “a user” means in the model

p_u is a vector attached to an *account*, and an account is not a person:

- A household shares one account, so p_u averages several tastes into one point — and the average of two tastes is often nobody’s
- A person’s taste changes; p_u is a single vector with no time in it. MovieLens spans years
- The same person on two devices is two users, with two independent vectors estimated from half the data each

WORTH SAYING BECAUSE THE MODEL CANNOT

Nothing in the objective represents any of this. A model whose unit of analysis is wrong will be confidently wrong, and no amount of held-out RMSE reveals it.

What people actually give

Rating	1	2	3	4	5
Share	5.6%	10.8%	26.1%	34.9%	22.6%

Mean 3.58, and heavily skewed upward: 57.5% of ratings are 4 or 5.

People mostly rate things they liked. Which means the rating scale is not being used as a scale — it is being used as “good”, “very good”, and occasionally “I want you to know this was bad”.

Why RMSE, and its quiet assumption

$$\mathrm{RMSE} = \sqrt{\frac{1}{|\Omega_{\text{test}}|} \sum_{(u,i)} (r_{ui} - \hat{r}_{ui})^2}$$

Squared error, so a two-star miss costs four times a one-star miss. Defensible, and worth stating: it says being badly wrong occasionally is worse than being slightly wrong often.

✗ The quiet assumption is bigger. RMSE averages over the observed test ratings — films these users chose to watch and rate. It says nothing about the films they did not, which is precisely the set a recommender operates on.

Lecture 22 replaces it for that reason. Today it is the right metric for the question “what would they rate it”, and the wrong one for “what should we show”.

The long tail

Top films by rating count	Share of all ratings
top 1%	8.7%
top 5%	28.3%
top 10%	44.4%
top 20%	65.2%
Films with fewer than 20 ratings	17.9%

The top 5% of films take 28.3% of the ratings. Most of the catalogue is barely rated at all.

✗ Two consequences to keep. First, a model can score well by learning popularity and nothing else — Lecture 22 measures exactly that. Second, the films where a recommendation would be *valuable* are the ones with least data.

The tail is where the value is

A recommendation is useful to the extent the user would not have found the item anyway.

- The top 1% of films hold 8.7% of ratings. Recommending those is easy, scores well, and tells the user nothing they did not know
- 17.9% of films have under twenty ratings. These are where a recommender could earn its place
- And they are exactly where every model in this lecture has least to work with

> **THE METRIC DOES NOT KNOW THIS**
>
> RMSE over held-out ratings, and even NDCG over held-out interactions, give full credit for a correct popular recommendation and the same credit for a correct obscure one. **Nothing measured in this lecture rewards usefulness.** Lecture 22 tries.

BASELINES

How much of a rating is the user, and how much the film

15 minutes

The baselines, before any model

Rule 2. Split 90/10 at random, fit on the training half, and predict:

Predictor	RMSE
The global mean, for everyone	1.1185
This user's mean rating	1.0367
This film's mean rating	0.9826
Both, as biases	**0.9109**

A single number for everybody scores 1.1185. Knowing *which film* takes it to 0.9826; knowing *who* and *which film* takes it to 0.9109.

No interaction between the two has been modelled yet. Nothing so far knows that *this* user likes *this kind* of film.

The split, and why it is stated first

- 90/10 on the ratings, at random, one seed, fixed before any model was fitted
- 900,189 ratings to fit, 100,020 to test
- Every number in this lecture uses *that* split — baselines, neighbourhoods, factorisations, the SVD disasters

A RANDOM SPLIT OF RATINGS IS NOT INNOCENT

It puts a user's future ratings in the training set and their past in the test set, which no deployed system ever gets. It is the standard protocol for rating prediction, it is convenient, and it is optimistic. **Lecture 22 measures exactly how optimistic.**

The bias model, written out

$$\hat{r}_{ui} = \mu + b_u + b_i$$

THE GLOBAL MEAN, PLUS THIS USER'S OFFSET, PLUS THIS FILM'S

Fitted by alternating closed forms, each regularised by the count — a user with three ratings should not be given a large offset:

$$b_u = \frac{\sum_{i \in I_u} (r_{ui} - \mu - b_i)}{|I_u| + \lambda}$$

That λ in the denominator is shrinkage, and it is the ridge penalty of Lecture 5 arriving in a new place: with few observations, pull the estimate toward zero.

What the biases learned

	Lowest	Highest
User offset b_u	-2.2156	+1.4690
Film offset b_i	-1.8545	+1.0546

The harshest user rates nearly 2.2 stars below the mean on everything, and the most generous 1.5 above. That spread is larger than most of what the rest of this lecture will add.

> **WHY THIS MATTERS MORE THAN IT LOOKS**
>
> A recommender that ignores biases spends its capacity re-learning that some users are generous and some films are popular — effects with nothing personal in them. Modelling them **explicitly and cheaply** frees the interesting part of the model for the interesting part of the problem.

Cold start, named now

A user with no ratings has no p_u to solve for; a film with no ratings has no q_i. The factorisation has nothing to say about either, and the failure is total rather than gradual.

- **New user** — fall back to popularity, or ask for a few ratings up front
- **New item** — fall back to content: genre, cast, description, an embedding of the synopsis
- **New system** — you have neither, which is why every recommender starts as a popularity list

Not examinable in detail, but the reason a real system is always a hybrid of a factorisation and something that works without history.

DERIVATION 21 · EXAMINABLE

Matrix factorisation, and the SVD it is not

35 minutes

Step 1 · What we are given

A matrix $R \in \mathbb{R}^{m \times n}$ with $m = 6,040$ users and $n = 3,706$ films, of which we observe a set Ω of entries:

$$|\Omega| = 1,000,209, \qquad \frac{|\Omega|}{mn} = 4.5\%$$

R is not a sparse matrix in the sense of Lecture 19, where the zeros were real zeros. Here the unobserved entries have **no value at all** — they are not zero, not the mean, and not anything.

Every difficulty in this derivation comes from that one sentence, and every wrong method comes from forgetting it.

Step 2 · The low-rank hypothesis

Assume tastes are simpler than the data: that there are d underlying factors — how much a film is a comedy, how dark, how old — and that a rating is how much the user wants each factor times how much the film has of it.

$$\hat{r}_{ui} = p_u \cdot q_i, \qquad p_u, q_i \in \mathbb{R}^d$$

Equivalently $\hat{R} = PQ^\top$ with $\operatorname{rank} \le d$. The parameter count falls from mn to $d(m+n)$, which for $d = 16$ is about a hundredth of the matrix.

✗ It is a *hypothesis*, not a fact. Its justification is that it predicts held-out ratings better than the alternatives, and we will check that rather than assert it.

Step 3 · Lecture 8 already solved this

The best rank-d approximation of a matrix in the Frobenius norm is its truncated SVD. Eckart–Young:

$$\min_{\text{rank}(X)\leq d} \|R - X\|_F \;=\; \|R - U_d \Sigma_d V_d^\top\|_F$$

So the answer looks like it is already in our hands: take the SVD of the ratings matrix, keep d singular values, read off P and Q.

ONE CONDITION, AND IT IS NOT MET

The Frobenius norm sums over **every** entry of R. To evaluate it — let alone minimise it — every entry must have a value, and 95.5% of ours do not.

Interlude · What Eckart–Young actually says

Worth being precise, because the derivation turns on it. The theorem states: *among all matrices of rank at most d, the truncated SVD minimises $\|R - X\|_F$.*

- It is a statement about **one given matrix R**
- The norm sums over every entry, with equal weight
- It says nothing about entries R does not have — the question is not defined

> **SO THE THEOREM IS NOT WRONG; IT IS BEING MISUSED**
>
> Filling the matrix does not extend the theorem to our problem. It **changes our problem** into one the theorem covers, and the two problems have different answers. The next slide prices the difference.

Step 4 · So fill them in?

The obvious repair: put something in the empty cells and take the SVD of the completed matrix. Two obvious choices, both actually tried, both measured on the same held-out ratings as before:

Rank-32 SVD of…	RMSE
the matrix filled with **zeros**	✗ **2.3274**
the matrix filled with the **global mean**	0.9920
predicting the global mean for everyone	1.1185
the bias model	0.9109

✗ Filling with zeros scores 2.3274 — more than twice the error of predicting one number for everybody, from the mathematically optimal rank-32 approximation.

Why zeros are worse than the mean

Rank-32 SVD of the matrix filled with…	RMSE
zeros	✗ 2.3274
the global mean	0.9920

Ratings run from 1 to 5, so a zero is not a low rating — it is **off the scale**. The approximation is dragged toward zero everywhere, and the predictions for observed pairs come out far too low.

Filling with the mean at least puts the fill inside the range, so the damage is bounded: 0.9920, which is merely worse than the bias model (0.9109) rather than catastrophic.

✗ Both are the same error. One is just more visible, and the more visible error is the safer one to make.

Step 5 · Why filling fails

The SVD did exactly what it promised: the best rank-32 approximation to **the matrix it was given**. That matrix is 95.5% zeros, so the best approximation is mostly a good approximation of the zeros.

- The fill value is not data. Optimising against it spends the model on reproducing an assumption
- And the assumption is a strong one: zero means “this user would rate this film 0”, on a scale that starts at 1
- Filling with the mean is less absurd — 0.9920 — and still worse than the bias model, which has $d = 0$

> THE TRANSFERABLE ERROR
>
> **Imputing missing values to make a method applicable makes the method answer a different question.** The same mistake as filling a missing feature with a column mean and then reporting the model as if the data had been complete.

Step 6 · Change the objective, not the data

The problem was never the SVD. It was that we asked for a fit to *every* entry. So ask only for what we can observe:

$$\min_{P,Q} \sum_{(u,i)\in\Omega} \left(r_{ui} - p_u \cdot q_i\right)^2$$

THE SUM RUNS OVER OBSERVED ENTRIES ONLY

The unobserved entries are now absent from the objective rather than assigned a value. Nothing is imputed, and nothing is assumed about what a user would have said.

✗ And in doing so we have left the world where the SVD applies. This objective has no closed form, no orthogonality, no unique solution — it is not an eigenproblem any more.

Step 6a · Read the objective once more

$$\min_{P,Q} \sum_{(u,i)\in\Omega} \left(r_{ui} - p_u \cdot q_i\right)^2 + \lambda \left(\|P\|_F^2 + \|Q\|_F^2\right)$$

- Ω — the observed entries, and only those. This is the whole departure from the SVD
- $p_u \cdot q_i$ — a rank-d model, written entrywise rather than as a matrix product, because the matrix product is over cells we do not have
- λ — without which a user with three ratings gets a perfect fit and no information

Three symbols, three decisions, and each one is a slide of this derivation. If you can reconstruct why each is there, you have the derivation.

Step 7 · What was given up

	Truncated SVD	This objective
Solution	closed form	iterative
Uniqueness	unique (up to sign and ties)	not unique — $PM, QM^{-\top}$ scores identically
Factors	orthogonal, ordered by variance	neither
Optimum	global	local — the problem is non-convex in (P, Q) jointly

All four losses are real, and the last two matter in practice: **you cannot interpret factor 3 the way you interpret principal component 3**, because a different run gives different, equally good factors.

Step 8 · Regularisation is not optional

A user with three ratings and $d = 16$ has more parameters than observations, so the objective can be driven to zero on that user while learning nothing.

$$\min_{P,Q} \sum_{(u,i)\in\Omega} \left(r_{ui} - p_u \cdot q_i\right)^2 + \lambda \left(\|p_u\|^2 + \|q_i\|^2\right)$$

Which is ridge regression again, from Lecture 5, and it is doing the same job: with few observations, prefer the smaller coefficients.

It also has a second job here. The penalty breaks the scaling degeneracy of the previous slide — $P \to 2P, Q \to Q/2$ is no longer free.

Step 9 · Hold one side fixed

The objective is not convex in P and Q together. But fix Q, and every term involving p_u is a quadratic in p_u alone:

$$\min_{p_u} \sum_{i \in I_u} (r_{ui} - p_u \cdot q_i)^2 + \lambda \|p_u\|^2$$

That is a **ridge regression**, with the films this user rated as the rows of the design matrix and their ratings as the target. And it is *separate* for each user: no user's solution depends on another's.

So the whole P step is m independent small regressions, each over only that user's own ratings.

Step 10 · The closed form

Write Q_u for the $|I_u| \times d$ matrix of the films user u rated, and r_u for their ratings. Setting the gradient to zero:

$$p_u = \left(Q_u^\top Q_u + \lambda I\right)^{-1} Q_u^\top r_u$$

The normal equation of Lecture 2, with the ridge term of Lecture 5. The system is $d \times d$ — 16×16 here — regardless of how many films the user rated.

✗ Note where the missing entries went: Q_u contains only the films this user rated. Nothing was imputed; the unobserved entries simply never enter the system.

Step 11 · Alternate

By symmetry, fixing P makes each q_i a ridge regression over the users who rated film i. So:

1. Fix Q; solve for every p_u
2. Fix P; solve for every q_i
3. Repeat

Each half-step **solves its subproblem exactly**, so neither can increase the objective. The objective is bounded below by zero, so the sequence converges.

Converges to a *local* minimum: monotone descent is not global optimality, and a different initialisation gives a different answer. That is the price of Step 6, and it was paid knowingly.

What an ALS sweep costs

Systems solved per sweep	6,040 users + 3,706 films
Size of each system	$d \times d$
At $d = 16$	16×16
Ratings gathered per sweep	900,189, twice

Each system is tiny and independent of the others, so the sweep is embarrassingly parallel and the cost is dominated by gathering each user's rows rather than by the linear algebra.

Which is why the notebook groups the ratings once with a single argsort instead of taking a boolean mask per user: the same arithmetic, and the difference between seconds and minutes.

Step 12 · Or descend, one rating at a time

The alternative, and the one that scales past memory. For each observed rating, take the error and step both vectors:

$$e_{ui} = r_{ui} - \hat{r}_{ui}$$

$$p_u \mathrel{+}= \eta \left(e_{ui} q_i - \lambda p_u \right)$$

$$q_i \mathrel{+}= \eta \left(e_{ui} p_u - \lambda q_i \right)$$

- **ALS** parallelises across users and each step is exact — but needs a $d \times d$ solve per user
- **SGD** touches one rating at a time, so it suits implicit data and streams — but needs a learning rate

Same objective, different route. At $d = 32$: ALS reaches 0.8733, SGD 0.8615.

SGD, watched converging

Epoch	1	2	3	4	5	6	7	8
train RMSE	0.9126	0.9012	0.8961	0.8916	0.8848	0.8750	0.8651	0.8551

Fifteen epochs of per-rating updates end at test RMSE 0.8615; twelve ALS sweeps at $d = 32$ reach 0.8733. Two routes to the same objective, arriving in the same place.

✗ Note the *training* RMSE keeps falling while we care about the test one. Nothing in the SGD loop knows the difference — which is Lecture 5 again, and the reason the epoch count is a capacity parameter here just as the rank is.

Step 13 · Put the biases back

The factorisation models the *interaction*. The parts that are not interactions — a generous user, a popular film — should not be spent on it:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$$

Fit the biases first, then factorise what they leave behind.

Bias model alone ($d = 0$)	0.9109
Factorisation, $d = 16$, with biases	0.8587
Factorisation, $d = 16$, no biases	0.8559

The biases alone close 79.9% of the distance from the global mean to the full model. Most of a rating is not personal.

Step 14 · An honest wrinkle

At $d = 16$, the model *without* explicit biases scores 0.8559 — slightly **better** than the one with them (0.8587).

The textbook claim is that biases help. Here they do not, and the reason is visible in the algebra: a factor dimension can hold a constant, so a rank-16 factorisation can *represent* the biases itself, using two of its sixteen dimensions to do it.

> **WHAT IS STILL TRUE**
>
> Biases are worth having because they are **cheap, stable and interpretable**, they help most at low rank, and they let the factors be about interaction. Not because they always lower RMSE — and the difference here, 0.0028, is small enough that reporting it as a finding either way would be overreaching.

Step 15 · How many factors?

d	1	2	4	8	16	32	64
RMSE	0.8912	0.8836	0.8705	0.8602	0.8587	0.8733	0.9006

Best at $d = 16$ (0.8587), and rising again by $d = 64$ (0.9006).

A U-curve, which is Lecture 5 exactly: more capacity fits the training ratings better and the held-out ratings worse. The rank is a capacity parameter, and it is chosen the way every capacity parameter in this course is chosen — by measurement on data the model did not see.

Even $d = 1$ scores 0.8912, already better than the bias model. One dimension of taste is worth something.

Step 16 · What the factors are not

- They are **not** principal components: not orthogonal, not ordered, not unique
- They are **not** genres. A run may put something genre-like in a dimension, and the next run will not put it in the same one
- They are **not** interpretable individually. Only the inner product is determined; any invertible M gives PM and $QM^{-\top}$ the same predictions

> **SAY THIS OUT LOUD WHEN SOMEONE SHOWS YOU A FACTOR PLOT**
>
> Reading meaning into an individual latent dimension of a factorisation is the recommender-systems version of reading a coefficient of a collinear regression. The model is not claiming what the plot claims.

Interlude · The rotation nobody can see

For any invertible M:

$$(PM)(QM^{-\top})^{\top} = PMM^{-1}Q^{\top} = PQ^{\top}$$

Every prediction is identical, so the objective cannot distinguish P from PM. The solution is a whole family, and an optimiser returns whichever member its initialisation led to.

- The **predictions** are determined — that is what we wanted
- The **factors** are not, and no amount of staring at them fixes that

The regularisation narrows the family — it penalises $\|P\|$, so M can no longer be an arbitrary scaling — but rotations of an isotropic penalty remain free.

LAST TWENTY MINUTES

Neighbourhoods, and feedback nobody gave

20 minutes

The other family: neighbourhoods

Before factorisation there was a simpler idea, and it is still deployed: predict from **similar things**.

- **User-based** — find users who rated as you did, and average their ratings for the film in question
- **Item-based** — find films rated as this film was, and average *this user's* ratings of them

Item-based is what gets deployed, for a reason that is operational rather than statistical: there are usually far fewer items than users, item–item similarities change slowly, and they can be computed offline and cached.

Item–item similarity, defined

$$s_{ij} = \frac{\tilde{r}_i \cdot \tilde{r}_j}{\|\tilde{r}_i\| \, \|\tilde{r}_j\|}$$

COSINE BETWEEN TWO FILMS' RATING COLUMNS

The tilde matters: the columns are **bias-adjusted** first, $\tilde{r}_{ui} = r_{ui} - \mu - b_u - b_i$. Without that, every popular film is similar to every other popular film, because both columns are full of fours.

✗ Which is the same trap as an unnormalised dot product in Lecture 20, and the same trap as term frequency without idf in Lecture 19: subtract what is true of everything before measuring what is specific.

The prediction, and the measurement

$$\hat{r}_{ui} = \mu + b_u + b_i + \frac{\sum_{j \in N_k(i)} s_{ij}\, \tilde{r}_{uj}}{\sum_{j \in N_k(i)} s_{ij}}$$

Method	RMSE
item–item, $k = 10$	0.9376
item–item, $k = 20$	0.9171
item–item, $k = 50$	0.8867
factorisation, $d = 16$	**0.8587**

The best neighbourhood model (0.8867) does not reach the factorisation, and still beats the bias model (0.9109).

Why factorisation wins, and where it does not

	Neighbourhood	Factorisation
Uses	direct co-rating evidence	a global low-rank structure
Sparse items	**✗ few co-ratings, poor estimate**	borrows strength from everything
Explainable	**“because you liked X” ✓**	**✗ a dot product of latent vectors**
New rating	update a cached similarity	refit, or fold in approximately

Row three is why neighbourhood methods survive: a recommendation a user can be *told the reason for* is worth points a metric does not award.

And in practice the two are combined, exactly as BM25 and dense retrieval were combined last lecture, and for the same reason.

Where a neighbourhood method runs out

Two films are similar only if enough users rated *both*. In the long tail there are no such users.

- 17.9% of films have fewer than twenty ratings at all
- For two such films, the co-rating count is typically zero, and the cosine of two nearly empty columns is noise
- A factorisation has no such problem: a tail film's q_i is estimated from its few ratings *and* from where those raters sit in a space learned from everything else

> **BORROWING STRENGTH**
>
> That is the real advantage of the low-rank model, and it is the same argument as pooling in a hierarchical model: a parameter estimated from three observations is rescued by a structure estimated from a million.

Explicit ratings are the easy case

Everything so far assumed a number the user typed. Most real systems have none.

Explicit	Implicit
a rating, deliberately given	a click, a play, a purchase, a dwell time
scarce — most users rate nothing	abundant — every interaction is one
a scale, with a middle	presence or absence
a stated preference	a revealed one, plus the interface

MovieLens is explicit, which is why it can be used to teach rating prediction at all. It is *not* representative: your first recommender will almost certainly have no ratings in it.

How much data implicit feedback is

Binarise our ratings at 4 or above and call that an interaction:

Explicit ratings	1,000,209
Of which positive (≥ 4)	575,281
As a fraction	57.5%
Cells with no interaction	22.4 million

In a real system the first row would be zero and the second would be a click log ten times larger. The proportions are what matter: a few hundred thousand positives against tens of millions of cells whose meaning is unknown.

This binarised view is what Lecture 22 uses throughout, which is why it is defined here.

Implicit feedback has no negatives

A user watched film A. What does that say about film B, which they did not watch?

- They disliked it — a true negative
- They have not seen it yet — possibly the best recommendation available
- They were never shown it — and that was the old system's choice, not theirs

> **WHICH IS THE WHOLE DIFFICULTY**
>
> In explicit data the missing entries are excluded from the objective, and we saw exactly what happens when they are not. In implicit data the **only** thing that exists is the positives, so an objective over observed entries alone is trivially satisfied by predicting "yes" for everything.

Two ways out

- **Weighted least squares over everything**: treat every unobserved cell as a zero with *low confidence*, and observed ones as one with confidence rising in the number of interactions. All 22.4 million cells enter the objective, with weights
- **Pairwise ranking**: never predict a value at all. Require only that an observed item outranks an unobserved one for that user

The first keeps the closed form — the weighting is arranged so that ALS still works — and needs a confidence function you have to justify. The second changes the objective into a ranking objective, which is what we actually want, and gives up the closed form.

BPR: rank the pair, not the item

$$\max \sum_{(u,i,j)} \log \sigma(\hat{r}_{ui} - \hat{r}_{uj}) - \lambda \|\Theta\|^2$$

I OBSERVED FOR U, J SAMPLED FROM THE UNOBSERVED

- Only the *difference* is modelled, so no absolute score is ever claimed — and none was ever observed
- j is **sampled**, not enumerated, which is what makes it affordable
- The logistic of a score difference is the pairwise logistic loss of Lecture 4, and the sampling is the negative sampling of Lecture 20

✗ And it inherits Lecture 20's trap exactly: a sampled "negative" may be an item the user would love. Being unobserved is not evidence of dislike.

Where we are

Model	RMSE	Knows
Global mean	1.1185	nothing
Film mean	0.9826	which film
Bias model	0.9109	who, and which film
SVD, mean-filled	✗ **0.9920**	an imputation it invented
Item–item, $k = 50$	0.8867	co-rating evidence
Factorisation, $d = 16$	**0.8587**	**a low-rank structure**

Every row measured on the same held-out 100,020 ratings, with the same split and the same seed.

What a deployed recommender actually is

1 Candidates

catalogue → hundreds. Factorisation, popularity, co-visitation, recency — several sources, unioned.

2 Rank

hundreds → tens. A richer model with features the first stage cannot afford.

3 Policy

diversity, freshness, business rules, exploration.

The same three stages as Lecture 19's search pipeline, for the same reason: the accurate model does not scale to the catalogue. What we built today is stage 1.

Stage 3 has no equation in it and decides more of what a user sees than stages 1 and 2 together.

The distance travelled, in one number

	RMSE	Distance closed
Global mean	1.1185	0%
Bias model	0.9109	79.9%
Item–item, $k = 50$	0.8867	89.2%
Factorisation, $d = 16$	**0.8587**	**100%**

Two arithmetic means and a shrinkage parameter get you most of the way. The derivation, the alternating solves and sixteen latent dimensions get the rest.

✗ That ratio is not an argument against the model. It is an argument for always fitting the baseline first, so you know which part of your result is the idea and which part is arithmetic.

Five questions for a recommender result

1. **Rating prediction or ranking?** RMSE and NDCG answer different questions and are not comparable
2. **What is the baseline?** The bias model is four lines and is often close
3. **How was it split?** Randomly over ratings, or in time?
4. **What happened to the missing entries?** Excluded, imputed, or weighted — and if imputed, with what
5. **Explicit or implicit**, and if implicit, where did the negatives come from?

Question three is the one Lecture 22 turns into a measurement.

Where this sits in the course

Lecture 2	the normal equation — here, once per user
Lecture 5	ridge, and a capacity U-curve — here, in the rank
Lecture 8	the SVD — here, as the method that does not apply
Lecture 19	ranking metrics — next lecture, replacing RMSE
Lecture 20	the bi-encoder — next lecture, with users as queries

Row three is the one to remember. A method you know well can be exactly wrong for a problem that looks like the one it solves, and the difference was a single word in the problem statement: *missing*.

The notebook for this lecture

Open Lecture 21 in Colab

- **Runs on free CPU** in a few minutes. ALS is a $d \times d$ solve per user, and there are only 6,040 of them
- The SVD-with-zeros disaster reproduced, so you see the number rather than take my word for it
- ALS derived and implemented from Step 10, and checked against a per-rating SGD written from Step 12

> **WHAT TO CHANGE**
>
> Set the regularisation to zero and re-run the rank sweep. Watch where the U-curve moves, and which rank stops being the best one.

Vocabulary

Explicit / implicit	a stated rating, against an observed interaction
Cold start	a user or item with no history to factorise
Bias term	the part of a rating that is not an interaction
Latent factor	a coordinate of p_u or q_i; individually meaningless
ALS	alternating ridge regressions, one side at a time
BPR	a pairwise ranking objective for implicit data
Long tail	the majority of items, holding a minority of the interactions

Where students lose marks on this material

- Saying “matrix factorisation is the SVD”. It is not, and being able to say *why* in one sentence is the most-asked examination question in this part
- Writing the objective as a sum over all (u, i) rather than over Ω
- Claiming ALS finds the global optimum. Each half-step is exact; the joint problem is not convex
- Interpreting a latent factor as a genre
- Treating an unobserved entry as a negative in implicit data — the error the whole last section is about

Reading

- **These lecture notes are the primary source** — lecture-21.pdf, the extended notes for this lecture. The textbook does not cover this material
- Koren, Bell and Volinsky, *Matrix Factorization Techniques for Recommender Systems* — the Netflix-era summary, and still the clearest account of biases
- Hu, Koren and Volinsky, *Collaborative Filtering for Implicit Feedback Datasets* — the weighted-ALS method
- Rendle et al., *BPR* — the pairwise objective

Examinable content is what is on these slides and in the notebook.

Scope

Scope

- the recommendation problem and how its data differs; missing-not-at-random; **Derivation 21** in full — the low-rank model, why the SVD cannot be taken, the observed-entry objective, the ALS half-step and its closed form, the SGD update, biases; the rank as a capacity parameter; item–item neighbourhoods; why implicit feedback has no negatives, and BPR's response — EXAMINABLE
- recommender libraries, serving infrastructure, incremental refitting — NOT EXAMINABLE — ENGINEERING
- SVD++ and time-aware factorisation, the full weighted-ALS derivation for implicit data, bandits for exploration — BEYOND THE SYLLABUS

Summary

1. Recommendation is retrieval with the query replaced by a history, and the labels are a by-product of use rather than a measurement
2. 95.5% of the matrix is missing, and missing is not zero — filling it and taking the SVD scores 2.3274, worse than predicting one number for everybody
3. Restrict the objective to observed entries and you leave the SVD behind: no closed form, no uniqueness, no interpretable factors — and a model that works
4. ALS is a ridge regression per user; SGD is one update per rating. Same objective, different route, comparable answers
5. Most of a rating is not personal. The biases alone reach 0.9109 against the full model's 0.8587

One line to keep

> THE LINE
>
> **The missing entries are the data. Impute them and you fit your own assumption.**

The SVD is optimal for the problem it is given, and giving it a filled-in matrix gives it the wrong problem — 2.3274 against the bias model's 0.9109. Changing the objective, not the data, is what the whole derivation was for.

Before the next lecture

- Run the notebook. Reproduce the zero-filled SVD number yourself — it is the most surprising figure in Part V and it takes four lines
- Next lecture keeps the model and changes the **question**: from “what would they rate it” to “what should we show”, which is a ranking problem and needs Lecture 19’s metrics
- It is also where the evaluation goes wrong. Come prepared to distrust every recommender result you have ever read, including the ones in these slides

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 21

five, in the style of the written paper

Exercises — Lecture 21

1 State why the truncated SVD cannot be applied directly to a ratings matrix, naming the condition of Eckart–Young that fails. [5]

2 A rank-32 SVD of the zero-filled matrix scores worse than predicting one number for everybody. Explain how the optimal approximation can lose to a constant. [5]

3 Write the ALS half-step for one user, and identify it as a familiar estimator. [5]

Solutions on Lecture 22’s deck.

Exercises — Lecture 21 (continued)

4 For any invertible M, $(PM)(QM^{-\mathsf{T}})^{\mathsf{T}} = PQ^{\mathsf{T}}$. State what this settles about interpreting a latent factor. [4]

5 Implicit feedback has no negatives. State why an objective over observed entries alone fails, and name the two standard responses. [5]

Solutions on Lecture 22's deck.

THE FIVE SET AT THE END OF LECTURE 20

Solutions Lecture 20

not part of this lecture

Solutions — Lecture 20

1. Explain why a bi-encoder can be indexed and a cross-encoder cannot, and say what the two differences have in…

✓ $E(d)$ does not depend on the query, so it can be precomputed; the cross-encoder reads the pair jointly. The same fact causes the accuracy difference.

- Whether the query is present when the document is read decides both
- Nothing can be stored for a function that does not exist until the query arrives

2. A first stage has recall@100 of 0.885. State the highest recall a perfect re-ranker over those 100 candidates…

✓ 0.885. Improving the first stage raises the ceiling; improving the second never does.

- A re-ranker reorders a fixed candidate set and cannot retrieve
- 11.5% of relevant documents are simply absent from the candidates

Solutions — Lecture 20 (2)

3. In-batch negatives are described as free. State precisely what is free and what is not, and why the batch…

✓ The B^2 scores are nearly free; the $2B$ encoder passes are not. The batch size is the number of negatives each query is scored against, so it changes the task.

- Encoding grows linearly in B and the dot products quadratically, so the scores cost almost nothing beside the encoder
- Doubling the batch doubles the negatives per query, which is a choice about the problem rather than about the hardware

4. Hard negatives are mined from a first stage's top results. State the trap, and why it is worse at…

✓ Many are relevant but unjudged, so training teaches the model that a correct answer is wrong.

- At evaluation an unjudged document is an accounting error
- At training it is a gradient pushing the model away from the right answer

Solutions — Lecture 20 (3)

5. An approximate index reports recall 0.80. State what that number must be measured against, and what is…

✓ Against the exact scan, never against the relevance judgements.

- Otherwise 'my index missed it' and 'my model ranked it low' become one number
- They have different fixes, so they must be measured separately

LECTURE 22 · LECTURE NOTES

Recommender systems: neural, and evaluated honestly

Two-tower models — Lecture 20’s bi-encoder, with users in place of queries
Negative sampling, and the sampled-softmax correction
Then evaluation: leave-one-out against a temporal split, sampled metrics and their bias, and popularity

Part V · Lecture notes · examinable